



WILLSIM

FICHE DE COURS · RESSOURCE PÉDAGOGIQUE GRATUITE

Initiation à l'IA pour l'ingénierie

Découvrir l'apprentissage automatique à partir de mesures de capteurs : préparer les données, prédire une grandeur physique, reconnaître l'état d'une machine et évaluer honnêtement un modèle, avec Python et scikit-learn.

NIVEAU

Débutant

DURÉE

6 h (3 séances de 2 h)

PRÉREQUIS

Bases de Python
(variables, boucles,
fonctions)

OUTILS

Python 3, NumPy,
pandas, scikit-learn,
Matplotlib, Jupyter

OBJECTIFS DE LA FICHE

À la fin de cette fiche, vous saurez :

- distinguer un problème de régression d'un problème de classification et nommer, dans un cas d'ingénierie, les caractéristiques et la cible ;
- charger un fichier CSV de mesures avec pandas, en retirer les valeurs manquantes ou aberrantes, puis le découper en jeux d'entraînement et de test ;
- entraîner avec scikit-learn une régression linéaire et deux classifieurs : les k plus proches voisins et l'arbre de décision ;
- évaluer un modèle avec la MAE, l'exactitude et la matrice de confusion, et repérer un surapprentissage grâce à la validation croisée.

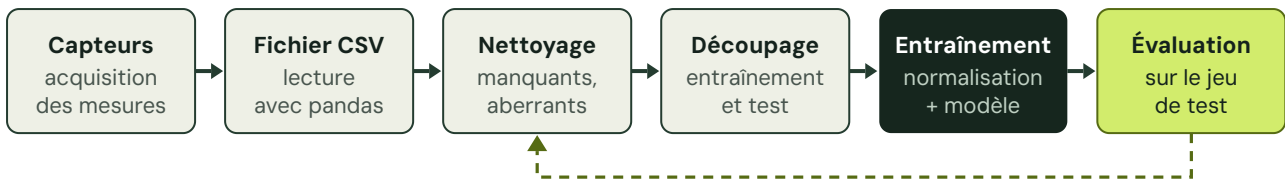
Déroulé conseillé : séance 1 → sections 1 et 2 ; séance 2 → sections 3 à 5 ; séance 3 → sections 6 et 7 (mini-projet).

1. Qu'est-ce que l'apprentissage automatique ?

Un programme classique applique des règles écrites par un humain, par exemple « si la température dépasse 80 °C, arrêter le moteur ». En **apprentissage automatique** (*machine learning*), on fournit au contraire des exemples : un algorithme ajuste lui-même les paramètres d'un **modèle** pour reproduire la relation entre des entrées et une sortie. Le modèle sert ensuite à prédire la sortie pour des données qu'il n'a jamais vues.

Une **observation** est une ligne du tableau de données (les mesures relevées à un instant) ; ses grandeurs d'entrée sont les **caractéristiques** (*features*, notées X) : courant, température, vibration... La **cible** (notée y) est la grandeur à prédire. Quand elle est connue pour chaque exemple d'entraînement, on parle d'**apprentissage supervisé** : c'est le cadre de cette fiche.

Si la cible est une valeur numérique continue, par exemple la température d'un moteur selon le courant et l'ambiante, c'est une **régression** (ici : régression linéaire). Si c'est une catégorie, comme l'état d'un moteur (normal, balourd ou surcharge), c'est une **classification** (ici : k plus proches voisins et arbre de décision).



Résultat insuffisant ? On améliore les données ou les caractéristiques, et on recommence.

Figure 1 — Chaîne de traitement d'un projet d'apprentissage supervisé. La normalisation s'ajuste après le découpage, sur le seul jeu d'entraînement.

Mise en place : `pip install numpy pandas scikit-learn matplotlib jupyterlab`, puis `jupyter lab`. Les extraits s'exécutent dans un notebook ou un script `.py` ; ceux qui portent le même nom de fichier se suivent. Sorties obtenues avec NumPy 1.24 et scikit-learn 1.3 : de légers écarts sont possibles avec d'autres versions.

2. Préparer des données de capteurs

Les mesures d'un banc d'essai, d'un automate ou d'un microcontrôleur sont souvent exportées en **CSV** : une ligne par observation, une colonne par grandeur. La bibliothèque pandas les charge dans un `DataFrame`, un tableau à colonnes nommées. Pour un fichier de tableau français (point-virgule, virgule décimale) :

```
pd.read_csv("mesures.csv", sep=";", decimal=",")
```

2.1 Nettoyer les mesures

- **Valeurs manquantes** (NaN, capteur débranché, trame perdue) : on supprime les lignes concernées (`dropna`) quand elles sont rares.
- **Valeurs aberrantes** (saturation, parasite) : la plage de mesure du capteur et les limites physiques du système sont les meilleurs critères de tri.
- **Unités** : une grandeur garde la même unité dans tout le fichier (pas de mélange A et mA).

L'extrait suivant simule 200 mesures d'un moteur en régime permanent, y introduit trois défauts, les enregistre en CSV, puis relit et nettoie le fichier. Avec des mesures réelles, seule la partie 2 du script sert.

regression_temperature.py – partie 1 : données

```
import numpy as np
import pandas as pd

# 1. Jeu de données synthétique : 200 mesures en régime permanent
rng = np.random.default_rng(42)          # graine fixe : résultats reproductibles
n = 200
df = pd.DataFrame({
    "t_ambiante": rng.uniform(15, 35, n),    # température ambiante (°C)
    "courant": rng.uniform(0.5, 2.5, n),      # courant absorbé (A)
})
# Échauffement dû aux pertes Joule : environ 8 °C/A² × I², plus un bruit de mesure
df["t_moteur"] = df["t_ambiante"] + 8 * df["courant"] ** 2 + rng.normal(0, 1.5, n)
df.loc[[3, 50], "t_moteur"] = np.nan        # deux mesures perdues
df.loc[7, "courant"] = 99.0                  # une valeur aberrante (capteur saturé)
df.to_csv("mesures_moteur.csv", index=False)

# 2. Lecture du fichier CSV et nettoyage
df = pd.read_csv("mesures_moteur.csv")
df = df.dropna()                            # supprime les lignes incomplètes
df = df[df["courant"].between(0, 5)]         # garde les valeurs physiquement plausibles
print(len(df), "lignes conservées")         # 197
print(df.describe().round(1))
```

2.2 Normaliser les caractéristiques

Le courant vaut quelques ampères, la température plusieurs dizaines de degrés : les algorithmes fondés sur des distances, comme les k plus proches voisins, seraient dominés par la grandeur la plus étendue. La **standardisation** ramène chaque caractéristique à une moyenne nulle et un écart-type de 1 :

$$z = \frac{x - \mu}{\sigma}$$

où μ et σ sont la moyenne et l'écart-type de la caractéristique, **calculés sur le jeu d'entraînement** (rôle de `StandardScaler`). La régression linéaire par moindres carrés et les arbres de décision n'en ont pas besoin : leurs prédictions n'en dépendent pas.

2.3 Découper en jeux d'entraînement et de test

Seule compte la performance sur des données nouvelles : avant tout apprentissage, on met de côté 20 à 30 % des observations, le **jeu de test**. `train_test_split` fait ce tirage au hasard, `random_state` le rend reproductible et `stratify=y` conserve la proportion de chaque classe.

ATTENTION · FUITE DE DONNÉES

Calculer μ et σ sur toutes les données avant le découpage fait « fuir » de l'information du test vers l'entraînement : l'évaluation devient trop optimiste. Ajustez toute transformation sur le seul jeu d'entraînement, via un `pipeline` (section 4). Pour des mesures successives d'un même essai, presque identiques, découpez par essai ou par période, pas au hasard.

3. Régression linéaire : prédire une grandeur physique

Une régression linéaire modélise la cible comme une somme pondérée des caractéristiques :

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots$$

L'entraînement choisit les coefficients w_i qui minimisent la somme des carrés des écarts entre mesures et prédictions (**méthode des moindres carrés**). Le script suivant prédit la température du moteur à partir de l'ambiante et du carré du courant.

regression_temperature.py – partie 2 : modèle (à la suite de la partie 1)

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
import matplotlib.pyplot as plt

X = pd.DataFrame({"t_ambiante": df["t_ambiante"],
                  "courant_carre": df["courant"] ** 2}) # caractéristique choisie : I²
y = df["t_moteur"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0) # 75 % entraînement, 25 % test
modele = LinearRegression().fit(X_train, y_train) # entraînement
y_pred = modele.predict(X_test) # prédictions sur le test
print("Coefficients :", modele.coef_.round(2))
print("Ordonnée à l'origine :", round(modele.intercept_, 2))
print(f"MAE sur le test : {mean_absolute_error(y_test, y_pred):.2f} °C")
plt.scatter(y_test, y_pred)
plt.plot([15, 80], [15, 80], "k--") # droite des prédictions parfaites
plt.xlabel("Température mesurée (°C)")
plt.ylabel("Température prédite (°C)")
plt.show()
```

Pourquoi I^2 ? En régime permanent, l'échauffement d'un moteur est à peu près proportionnel à la puissance qu'il dissipe en chaleur ; quand les pertes Joule dominent ($P = R \cdot I^2$), il suit donc le carré du courant. Avec la caractéristique I^2 , le modèle reste linéaire par rapport à ses coefficients, mais capture une relation physique non linéaire : une caractéristique bien choisie vaut souvent mieux qu'un algorithme plus complexe.

Le script affiche les coefficients `[1.03 7.99]`, une ordonnée à l'origine de $-0,75$ et une MAE de $1,13$ °C. Le modèle retrouve donc les valeurs de la simulation : environ 1 pour la température ambiante et 8 °C/A² pour I^2 . Cette erreur correspond au bruit de mesure simulé (écart-type de $1,5$ °C, soit une erreur absolue moyenne attendue d'environ $1,2$ °C) : aucun modèle ne fait mieux que ses mesures. Vérifier que les coefficients ont un sens physique est un excellent test de cohérence.

4. Classification : reconnaître l'état d'une machine

Le jeu de données simulé décrit un moteur dans trois états : **normal**, **balourd** (masse mal répartie sur le rotor : les vibrations augmentent) et **surcharge** (couple résistant excessif : le courant, puis la température, augmentent). Caractéristiques : vitesse vibratoire efficace (mm/s), température du carter (°C) et courant (A). Valeurs choisies pour l'exemple, non normatives, et tirées indépendamment selon une loi normale : c'est une simplification.

etat_moteur.py – partie 1 : jeu de données

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7) # graine fixe
# Moyennes (vibration en mm/s, température en °C, courant en A) de chaque état
moyennes = {"normal": [2.0, 50, 1.2], "balourd": [3.6, 53, 1.3], "surcharge": [2.4, 70, 2.1]}
ecarts_types = [0.6, 6, 0.3]
blocs = []
for etat, m in moyennes.items(): # 150 observations par état
    bloc = pd.DataFrame(rng.normal(m, ecarts_types, size=(150, 3)),
                        columns=["vibration", "temperature", "courant"])
    bloc["etat"] = etat
    blocs.append(bloc)
df = pd.concat(blocs, ignore_index=True) # 450 observations
```

4.1 Les k plus proches voisins (k-NN)

Pour classer une nouvelle observation, l'algorithme cherche les **k** observations d'entraînement les plus proches (distance euclidienne) et retient la classe majoritaire parmi elles. Il ne calcule aucun paramètre : il mémorise les données. Un petit k donne des frontières irrégulières, sensibles au bruit ; un grand k, des frontières plus lisses. La normalisation est indispensable.

4.2 L'arbre de décision

Un arbre pose une suite de questions de la forme « vibration $\leq 2,6$ mm/s ? » jusqu'à une feuille qui donne la classe. À chaque nœud, l'algorithme choisit la question qui sépare le mieux les classes (impureté de Gini, critère par défaut de scikit-learn). Un arbre peu profond se lit comme des règles, ce qui facilite le dialogue avec la maintenance. `max_depth` limite sa profondeur.

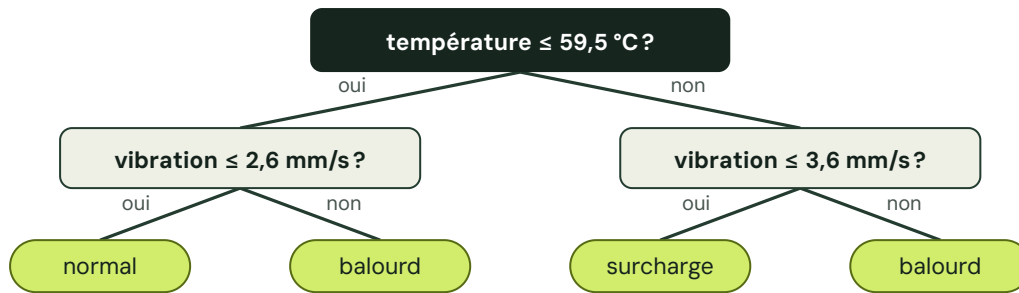


Figure 2 — Arbre de profondeur 2 appris sur le jeu d'entraînement de `etat_moteur.py` (seuils arrondis). Il se lit comme deux règles de diagnostic.

`etat_moteur.py` – partie 2 : entraîner et tester deux classifieurs

```

from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

X, y = df[["vibration", "temperature", "courant"]], df["etat"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=0) # mêmes proportions d'états
knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
arbre = DecisionTreeClassifier(max_depth=3, random_state=0)
for nom, modele in [("k-NN", knn), ("Arbre", arbre)]:
    modele.fit(X_train, y_train) # entraînement
    y_pred = modele.predict(X_test) # prédictions sur le test
    print(nom, "- exactitude :", round(accuracy_score(y_test, y_pred), 3))
    print(confusion_matrix(y_test, y_pred, labels=["normal", "balourd", "surcharge"]))
  
```

`make_pipeline` enchaîne normalisation et classifieur : μ et σ sont calculés pendant `fit`, sur le seul jeu d'entraînement. Exactitudes sur le test, données ici pour illustrer les métriques : 0,938 pour le k-NN, 0,903 pour l'arbre.

5. Évaluer un modèle : les métriques

5.1 En régression : l'erreur absolue moyenne (MAE)

$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$ est la moyenne des écarts absolus entre mesures et prédictions. Exprimée dans l'unité de la cible (ici en °C), elle se compare directement à la précision du capteur ou à la tolérance exigée.

5.2 En classification : exactitude et matrice de confusion

L'**exactitude** (*accuracy*) est la proportion de prédictions correctes. Elle peut tromper : si 99 % des mesures viennent d'un moteur sain, répondre toujours « normal » donne 99 % sans détecter un seul défaut. La **matrice de confusion** détaille les erreurs : dans scikit-learn, une ligne par classe réelle, une colonne par classe prédite.

Matrice de confusion du k-NN (jeu de test)

Réel \ Prédit	normal	balourd	surcharge
normal	36	2	0
balourd	4	34	0
surcharge	0	1	36

La diagonale compte les bonnes réponses : 106 sur 113, soit 0,938. Six des sept erreurs mélangent « normal » et « balourd », dont les plages de vibration se recouvrent. Un balourd non détecté (4 cas) n'a pas le même coût qu'une fausse alarme (2 cas) : on examine donc toujours la matrice entière. `classification_report` en déduit, classe par classe, la **précision** et le **rappel**.

6. Surapprentissage et validation croisée

Un modèle trop flexible peut apprendre par cœur le bruit des données d'entraînement : excellent sur celles-ci, il se trompe sur des données nouvelles. C'est le **surapprentissage** (*overfitting*), signalé par un grand écart entre score d'entraînement et score de validation. À l'inverse, un modèle trop simple échoue partout : c'est le **sous-apprentissage**.

Pour choisir et régler un modèle (k-NN ou arbre, valeur de k, profondeur...) sans toucher au jeu de test, on utilise la **validation croisée à K blocs** : on découpe le jeu d'entraînement en K blocs, on entraîne K fois le modèle sur K - 1 blocs et on l'évalue sur le bloc restant, chaque bloc servant une fois à l'évaluation. La moyenne des K scores estime la performance, leur écart-type sa stabilité.

Arbres de décision de profondeur croissante, données du script `etat_moteur.py` : exactitude en apprentissage et validation croisée (moyenne ± écart-type de `cross_val_score(arbre, X_train, y_train, cv=5)`)

max_depth	1	2	3	5	None (illimitée)
Apprentissage	0,65	0,91	0,93	0,96	1,00
Validation croisée	0,63 ± 0,04	0,87 ± 0,06	0,90 ± 0,04	0,90 ± 0,05	0,90 ± 0,05

Avec une seule question, l'arbre ne donne que deux réponses pour trois états : c'est du sous-apprentissage. Au-delà de la profondeur 3, le score d'apprentissage monte jusqu'à 1,00 quand la validation croisée plafonne autour de 0,90 : les nœuds ajoutés apprennent le bruit, pas l'état du moteur. À performance égale, on retient le modèle le plus simple : ici, la profondeur 3.

À RETENIR

- Un modèle ne vaut que par ses données : mesures propres, unités cohérentes, caractéristiques ayant un sens physique (I^2 pour l'échauffement).
- Le jeu de test est mis de côté avant tout apprentissage et ne sert qu'une fois, à la fin ; les transformations (normalisation...) s'ajustent sur l'entraînement seul, via un `Pipeline`.
- Régression : MAE, dans l'unité de la grandeur. Classification : exactitude et matrice de confusion.
- Un grand écart entre score d'entraînement et validation croisée signale un surapprentissage. À score égal, préférez le modèle le plus simple.

7. Mini-projet : classer l'état d'un moteur

EXERCICE GUIDÉ · CHOISIR ET JUSTIFIER UN CLASSIFIEUR

Repartez des données et du découpage de `etat_moteur.py` ; n'utilisez pas le jeu de test avant l'étape 4.

1. **Explorer.** Affichez `df.groupby("etat").mean()` et un nuage de points vibration-température coloré par état. Quelle caractéristique distingue chaque état ?

2. **Régler le k-NN.** Pour $k \in \{1, 3, 5, 11, 21\}$, estimez par validation croisée à 5 blocs l'exactitude du pipeline `StandardScaler` + k-NN. Retenez le meilleur k.
3. **Comparer.** Idem pour l'arbre, `max_depth` de 1 à 6. Quelle famille retenez-vous, et pourquoi ?
4. **Évaluer une seule fois.** Entraînez le modèle retenu sur tout le jeu d'entraînement, puis affichez son exactitude et sa matrice de confusion sur le jeu de test.
5. **Tester la normalisation.** Passez le courant en mA (colonne $\times 1000$) et comparez en validation croisée le k-NN avec et sans `StandardScaler`. Expliquez l'écart.

Critères de réussite :

- le script s'exécute sans erreur et redonne les mêmes résultats ; le jeu de test ne sert qu'à l'étape 4 ;
- le modèle retenu atteint au moins 0,90 en validation croisée, et l'écart avec le score de test est commenté ;
- la confusion la plus fréquente et l'effet de l'unité du courant sont expliqués.

Indices : `cross_val_score` (module `sklearn.model_selection`) s'emploie comme dans le tableau de la section 6, avec le pipeline comme modèle. Nuage de points : un `plt.scatter(..., label=etat)` par groupe de `df.groupby("etat")`, puis `plt.legend()`. Ordres de grandeur : 0,90 à 0,94 selon k pour le k-NN normalisé ; 0,57 à 0,65 sans normalisation avec le courant en mA : ses écarts, de plusieurs centaines de mA, écrasent dans les distances ceux de la vibration (de l'ordre du mm/s), alors que le courant ne distingue que la surcharge.

ATTENTION · PASSER À UN MOTEUR RÉEL

Un modèle de diagnostic aide à décider, il ne remplace jamais les protections d'une machine (arrêt d'urgence, disjoncteur, protection thermique). Pour mesurer sur un vrai moteur : alimentation coupée avant toute intervention sur le câblage ou les capteurs, moteur solidement fixé, pièces tournantes sous carter.

Pour aller plus loin

- [Tutoriel officiel de Python \(en français\)](https://docs.python.org/fr/3/tutorial/) — <https://docs.python.org/fr/3/tutorial/>
- [pandas — prise en main](https://pandas.pydata.org/docs/getting_started/index.html) — https://pandas.pydata.org/docs/getting_started/index.html
- [scikit-learn — guide de l'utilisateur](https://scikit-learn.org/stable/user_guide.html) — https://scikit-learn.org/stable/user_guide.html
- [scikit-learn — pièges courants, dont la fuite de données](https://scikit-learn.org/stable/common_pitfalls.html) — https://scikit-learn.org/stable/common_pitfalls.html
- [Matplotlib — tutoriels](https://matplotlib.org/stable/tutorials/index.html) — <https://matplotlib.org/stable/tutorials/index.html>
- [Documentation de Jupyter](https://docs.jupyter.org/en/latest/) — <https://docs.jupyter.org/en/latest/>
- [Page du cours sur willsim.fr](https://willsim.fr/cours/ia-ingenierie-initiation.html) — <https://willsim.fr/cours/ia-ingenierie-initiation.html>

Fiche rédigée par l'équipe pédagogique WILLSIM, association loi 1901 (RNA W842013616) qui promeut l'usage de l'intelligence artificielle dans l'ingénierie et la mécanique. Version 1.0 – octobre 2026.

© 2026 WILLSIM — Licence Creative Commons BY-NC-SA 4.0 (creativecommons.org/licenses/by-nc-sa/4.0/deed.fr) : vous pouvez copier, diffuser et adapter cette fiche à des fins non commerciales, en citant WILLSIM et en partageant vos adaptations sous la même licence.

Une erreur, une suggestion ? Écrivez à contact@willsim.fr · willsim.fr